

Tag 7

Fehler finden & die while-Schleife

Python-Grundlagen-Schulung

Was du heute lernst

1. Fehler **lesen, verstehen** und **reparieren** (Debugging)
2. Was ist eine **while-Schleife**?
3. Wie stoppe ich eine Schleife gezielt mit **break** und **continue**?
4. Viel Übung mit eigenen kleinen Programmen

Teil 1

Der Logik-Sanitäter

Keine Angst vor Fehlermeldungen!

Eine Fehlermeldung ist **kein Weltuntergang**.

Sie ist wie ein **Wegweiser**:

Python sagt dir genau, *wo* und *warum* etwas nicht passt.

Heute übst du, diese Wegweiser lesen zu lernen.

Zwei Arten von Fehlern

1. Der Absturz

Python bricht sofort ab und zeigt eine rote Fehlermeldung.

→ Leicht zu finden, weil Python selbst dich darauf hinweist.

2. Der stille Fehler

Das Programm läuft ganz normal durch –

liefert aber ein **falsches** Ergebnis.

→ Viel tückischer, weil nichts "rot aufblinkt"!

Fehler 1: Die verschobene Einrückung

```
if speicher_voll:  
print("Warnung: Speicher voll!")
```

✗ Python meckert: **IndentationError**

Der Text nach `if:` muss **eingerrückt** sein –
sonst weiß Python nicht, was zum `if` gehört.

So wird es repariert

```
speicher_voll = True

if speicher_voll:
    print("Warnung: Speicher voll!")
    print("Bitte Backup starten.")
```

✓ Beide Zeilen sind **gleich weit** eingerückt –
beide gehören zum `if`-Block.

Fehler 2: Der fehlende Doppelpunkt

```
status = "offline"

if status == "offline"
    print("Server ist offline.")
```

✗ SyntaxError

Nach `if ...` (und auch nach `else`) muss
immer ein **Doppelpunkt** `:` stehen.

So wird es repariert

```
status = "offline"

if status == "offline":
    print("Server ist offline.")
else:
    print("Server läuft.")
```

✓ Jetzt stehen die Doppelpunkte an der richtigen Stelle.

Fehler 3: Ein Gleichheitszeichen zu wenig

```
port = 80  
  
if port = 80:  
    print("Webserver aktiv.")
```

✗ SyntaxError

= heißt "**weise zu**" (eine Schublade befüllen)

== heißt "**vergleiche**" (sind zwei Dinge gleich?)

So wird es repariert

```
port = 80  
  
if port == 80:  
    print("Webserver aktiv.")
```

✅ Zum **Vergleichen** brauchst du immer **zwei** Gleichheitszeichen.

Fehler 4: Der stille Logik-Fehler

Ziel: Zugriff erlauben, wenn der Name "Admin" oder "Superuser" ist.

```
username = "Superuser"

if username == "Admin" and username == "Superuser":
    print("Zugriff erlaubt.")
else:
    print("Zugriff verweigert.")
```

⚠️ Kein Absturz – aber das Ergebnis ist **falsch**!

Warum ist das falsch?

`and` bedeutet: **beides gleichzeitig** muss stimmen.

Eine Variable kann aber niemals **gleichzeitig**

`"Admin"` **und** `"Superuser"` sein!

Die Bedingung ist deshalb **immer False**.

Richtig wäre: `or` – "das eine **oder** das andere"

So wird es repariert

```
username = "Superuser"

if username == "Admin" or username == "Superuser":
    print("Zugriff erlaubt.")
else:
    print("Zugriff verweigert.")
```

✓ Mit `or` reicht es, wenn **eine** der beiden Bedingungen stimmt.

Fehler 5: Text und Zahl vergleichen

```
ping = input("Ping in ms eingeben: ")  
  
if ping < 50:  
    print("Verbindung ist schnell.")
```

✗ **TypeError**

`input()` liefert **immer Text** zurück –
auch wenn der Benutzer "45" eintippt!

Text und Zahl kann Python nicht der Größe nach vergleichen.

So wird es repariert

```
ping = int(input("Ping in ms eingeben: "))  
  
if ping < 50:  
    print("Verbindung ist schnell.")
```



`int(...)` wandelt den eingegebenen Text
in eine echte Zahl um.

Fehler 6: Doppelt verneint

Ziel: Patchen, wenn der Server **nicht** gesperrt ist.

```
gesperrt = False  
  
if not gesperrt == False:  
    print("Patch-Vorgang startet.")
```

⚠ Auch kein Absturz – aber verwirrend und falsch!

Warum ist das falsch?

`gesperrt ist False` → `gesperrt == False` ist bereits **True**

Das zusätzliche `not` davor dreht dieses True wieder zu **False** um.

Das ist das **Gegenteil** von dem, was gewollt war!

So wird es repariert

```
gesperrt = False

if not sperrt:
    print("Patch-Vorgang startet.")
else:
    print("Server ist sperrt!")
```



not sperrt reicht völlig aus –
keine doppelte Verneinung nötig.

Deine Aufgabe jetzt

Alle 6 Fehler warten im Colab-Notebook auf dich –
teilweise auskommentiert, damit sie den Code nicht blockieren.

Lies die Fehlermeldung, überlege dir die Ursache,
und repariere den Code selbst.

Teil 2

Die while-Schleife

Warum brauchen wir Schleifen?

Willst du 4-mal "Ping gesendet" ausgeben,
kannst du die Zeile 4-mal abtippen.

Aber was, wenn es **400** oder **40.000** Mal sein sollen?

Oder du weißt vorher **gar nicht**, wie oft –

z. B.: "Frag so lange nach dem Passwort, bis es stimmt."

Genau dafür gibt es Schleifen.

Bild: Der Wachmann an der Schranke

Ein Wachmann hat eine einzige Regel:

"Solange noch Autos warten, lasse das nächste durch."

Er prüft das **nicht nur einmal**,
sondern vor **jedem einzelnen Auto neu**.

Erst wenn keine Autos mehr da sind, hört er auf.

Genau das macht while

```
zaehler = 1

while zaehler <= 3:
    print(f"Durchlauf Nummer: {zaehler}")
    zaehler = zaehler + 1
```

Vor **jedem** Durchlauf wird neu geprüft:
"Ist die Bedingung noch wahr?"

Schauen wir uns das Schritt für Schritt an →

Durchlauf 1

Ist `zaehler <= 3` ? `zaehler` ist **1** → `1 <= 3` → **Ja!**



Ausgabe: "Durchlauf Nummer: 1"



`zaehler` wird um 1 erhöht → jetzt **2**

Durchlauf 2

Ist `zaehler <= 3` ? `zaehler` ist **2** → `2 <= 3` → **Ja!**



Ausgabe: "Durchlauf Nummer: 2"



`zaehler` wird um 1 erhöht → jetzt **3**

Durchlauf 3

Ist `zaehler <= 3` ? `zaehler` ist 3 → `3 <= 3` → **Ja!**



Ausgabe: "Durchlauf Nummer: 3"



`zaehler` wird um 1 erhöht → jetzt 4

Die Prüfung, die alles beendet

Ist `zaehler <= 3` ? `zaehler` ist `4` → `4 <= 3` → **Nein!**



Die Schleife stoppt.

Das Programm läuft direkt nach der Schleife weiter.

Der wichtigste Satz zur while-Schleife

Ohne `zaehler = zaehler + 1`

würde die Schleife NIE aufhören!

Diese Zeile ist **kein Extra** – sie ist der Grund, warum die Schleife irgendwann endet.

Achtung: Die Endlosschleife

Wenn die Bedingung **niemals** `False` wird,
läuft die Schleife **für immer**.

```
while True:  
    print("Ich laufe für immer!")
```

Keine Panik!

Passiert dir das aus Versehen in Colab:

☞ Klick einfach auf den **Stop-Button**
("Interrupt execution") neben der Zelle.

Das ist **kein Schaden** am System –
nur eine Berechnung, die du stoppen musst.

Ist while True: immer ein Fehler?

Nein! `while True:` ist ein bewusst genutztes, sehr gebräuchliches Muster.

Man nutzt es genau dann, wenn man die Schleife **von innen heraus** beenden möchte.

Wie das geht, lernst du jetzt: mit **break**!

Teil 3

break und continue

break: der Notausschalter

`break` beendet die Schleife **sofort und komplett** –
egal was die Bedingung am Kopf sagt.

Das Programm springt direkt zum Code **nach** der Schleife.

Beispiel: das unendliche Login

```
while True:
    eingabe = input("Passwort eingeben: ")
    if eingabe == "admin123":
        print("Schleife wird beendet.")
        break
    print("Falsches Passwort!")
```

`while True` heißt: "wiederhole für immer".

Aber die `if` -Bedingung **innerhalb** entscheidet,
wann `break` ausgelöst wird – und **dann** ist Schluss.

continue: der sanfte Sprung

`continue` beendet **nicht** die ganze Schleife –
nur den **aktuellen Durchlauf** wird abgebrochen,
die Schleife läuft danach ganz normal weiter.

Bild: Lied überspringen

`break` = die **ganze Playlist** stoppen 

`continue` = nur **diesen einen Song** überspringen,
die Playlist läuft weiter 

Beispiel: eine Zahl überspringen

```
zahl = 0
while zahl < 5:
    zahl = zahl + 1
    if zahl == 3:
        continue
    print(f"Zahl: {zahl}")
```

Ausgabe: 1, 2, 4, 5

Die 3 fehlt – bei ihr wurde `continue` ausgelöst,
und das `print` für diesen Durchlauf übersprungen.

Merkregel

break = Schleife komplett verlassen

**continue = diesen Durchlauf überspringen,
aber weitermachen**



Kurze Pause

Teil 4

Jetzt du: der große Übungsblock

Was dich erwartet

Kategorie A – Einfache Schleifen

Countdown, Fortschrittsbalken, Ping-Zähler

Kategorie B – mit break und continue

Login-Versuche, Ports überspringen, Chatbot

Kategorie C – Komplexere Aufgaben

Temperatur-Simulator, kleiner Task-Manager

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Ein Beispiel zum Warmwerden

```
counter = 5
while counter > 0:
    print(counter)
    counter = counter - 1

print("Patchvorgang startet...")
```

Ein einfacher Countdown – genau das Prinzip,
das du gerade Schritt für Schritt kennengelernt hast.

Was du heute gelernt hast

- ✓ Fehlermeldungen sind **Hinweise**, keine Katastrophen
- ✓ `while` prüft die Bedingung **vor jedem** Durchlauf neu
- ✓ `break` verlässt die Schleife komplett, `continue` überspringt nur einen Durchlauf
- ✓ Eine Endlosschleife ist mit dem Stop-Button jederzeit sicher zu stoppen

Morgen: Entscheidungen & Funktionen 🎉